REST API Design Guidelines

Chapter 1. Resources and naming

A REST API models the domain as resources identified by URLs. Use plural nouns for collections (/users, /orders) and never verbs: the HTTP method already expresses the action. A concrete item lives under its collection, as in /users/42, and a nested relationship reads naturally as /users/42/orders.

Keep URLs lowercase, use hyphens rather than underscores for multi-word segments (/purchase-orders), and avoid exposing internal database identifiers when a stable business key exists.

Choose the HTTP method by semantics: GET to read, POST to create in a collection, PUT to replace a whole resource, PATCH for partial updates, and DELETE to remove. A well-designed API returns 201 Created with a Location header after a successful POST, and 204 No Content after a successful DELETE.

REST API Design Guidelines

Chapter 2. Versioning, pagination, and filtering

APIs evolve, and breaking changes must not surprise existing clients. The most common strategy is URL versioning, prefixing routes with /v1/. An alternative is header-based versioning through content negotiation, which keeps URLs stable but is harder to test from a browser. Whichever you choose, additive changes (new optional fields) should never require a version bump.

Collections need pagination. Offset pagination (?limit=20&offset=40) is simple and supports jumping to arbitrary pages, but degrades on large tables and can skip or duplicate items when rows are inserted mid-scan. Cursor pagination returns an opaque token pointing at the last seen item; it is stable under concurrent writes and fast at any depth, at the cost of not supporting random page access.

Expose filtering and sorting as query parameters (?status=active&sort=-created_at) and document which fields are indexed, so clients understand the performance envelope.

REST API Design Guidelines

Chapter 3. Error handling and security basics

Errors deserve the same design care as successes. Return the correct status code class (4xx for client mistakes, 5xx for server faults) and a machine-readable JSON body. The RFC 7807 'problem details' format standardizes this body with fields such as type, title, status, and detail, so clients can branch on error type instead of parsing human-readable messages.

Validation errors should list every failing field in one response rather than failing one field at a time. Rate limiting should answer 429 Too Many Requests together with a Retry-After header.

On security: always serve the API over HTTPS, authenticate requests with short-lived bearer tokens, and never leak stack traces or internal identifiers in error bodies. Idempotency keys on POST endpoints let clients retry safely after a network timeout without creating duplicate resources.